🚆 CAPSTONE PROJECT

Online Railway Reservation System (PrimeFaces Full Stack UI)

🎯 Objective

Design and develop a **complete railway reservation web application UI** using PrimeFaces integrated with backend services via
REST.

The system must support **end-to-end booking**, **real-time calculations**, **dynamic UI updates**, **advanced components**, and **performance optimization**, while also demonstrating **migration capability from a legacy JSF UI**.

🧩 Functional Requirements

1. 🔍 Train Search & Availability

Create a search interface with the following fields:

- Source Station (dropdown)
- Destination Station (dropdown)
- Journey Date (calendar)
- Class (Sleeper / AC / First Class)
- Quota (General / Tatkal / Senior Citizen)
- Flexible Date (checkbox)

Output:

Display results in a **DataTable** with:

- Train Number
- Train Name
- Departure / Arrival Time
- Duration
- Available Seats
- Base Fare
- Dynamic Status (Available / Waiting List / RAC)

👉 Must support:

- Pagination
- Sorting
- Filtering
- Lazy loading

- AJAX-based refresh

---

2. 👤 Passenger Management (Dialog-Based)

Use a **Dialog component** to manage passengers.

Fields per passenger:

- Full Name (min 3 chars)
- Age (1–120)
- Gender
- Berth Preference
- ID Type (Aadhar / Passport)
- ID Number
- Senior Citizen (checkbox)
- Special Needs (checkbox)

Rules:

- Maximum 6 passengers per booking
- Children under 5 → no seat allocation
- Senior citizens → discount applied

👉 Must support:

- Add / Edit / Delete (CRUD)
- Inline editing inside DataTable
- Real-time validation messages

---

3. 🎫 Booking & Fare Calculation

When a train is selected, dynamically calculate fare.

Fare Formula:

Total Fare=(Base Fare×Passengers)+GST+Service Charge+Tatkal Charge−Discount\text{Total Fare} = (\text{Base Fare} \times \text{Passengers}) + \text{GST} + \text{Service Charge} + \text{Tatkal Charge} - \text{Discount}Total Fare=(Base Fare×Passengers) +GST+Service Charge+Tatkal Charge−Discount

Detailed Rules:

- Base Fare varies by class & distance
- GST = 5% of total fare
- Service Charge = ₹20 per passenger
- Tatkal:
  - Sleeper → ₹100
  - AC → ₹200
- Senior Citizen Discount:

        - 40% reduction
    - Child (<5 years):
        - No fare

👉 Must:

- Update fare instantly (AJAX)
- Show breakdown in UI panel

---

4. 💳 Payment Module

Fields:

- Payment Method (Card / UPI / Net Banking)
- Card Number (16 digits validation)
- CVV (3 digits)
- Expiry Date
- Billing Address

Validations:

- Mandatory fields
- Format checks
- Prevent invalid transactions

👉 Show:

- Final payable amount
- Confirmation dialog

---

5. 📊 Admin Dashboard (Advanced UI)

Build a dashboard using advanced components:

Include:

- Charts:
    - Daily bookings
    - Revenue trends
- Summary cards:
    - Total bookings
    - Revenue
    - Cancelled tickets
- File Upload:
    - Upload train schedule (CSV)
- Wizard:
    - Multi-step booking flow

---

6. 🔄 REST API Integration

Integrate backend APIs:

- GET `/api/trains`
- GET `/api/availability`
- POST `/api/book`
- GET `/api/dashboard`

Requirements:

- JSON binding
- Error handling (invalid responses)
- Loading indicators during API calls
- Retry mechanism for failures

---

7. ⚡ AJAX & Dynamic Behavior

Implement:

- Partial page updates (no full refresh)
- Loading spinner during operations
- Dynamic seat availability refresh
- Real-time fare calculation

---

8. 🎨 UI Layout & Theming

- Use layouts:
    - Header (menu)
    - Sidebar navigation
    - Content area
- Apply PrimeFaces theme
- Ensure responsive UI (mobile-friendly)

---

9. 🔔 Validation & User Feedback

Must include:

- Field-level validation messages
- Global notifications (`growl`)
- Error messages from API
- Success confirmations

---

10. 🔁 Migration Requirement

Take one **sample legacy RichFaces UI (given or assumed)** and:

- Convert it to PrimeFaces
- Document:
    - Component mapping
    - AJAX differences
    - UI improvements
    - Performance gains

---

11. ⚙️ Performance Optimization

- Use lazy loading in DataTable
- Reduce unnecessary AJAX calls
- Optimize resource loading
- Avoid large DOM rendering

---

12. 🧪 Testing Requirements

- Manual UI testing
- Validation testing
- API failure scenarios
- Edge cases:
    - Invalid inputs
    - No seat availability
    - Payment failure

---

13. ♿ Accessibility & Standards

- Proper labels for inputs
- Keyboard navigation support
- Readable color contrast
- Clean, maintainable code structure

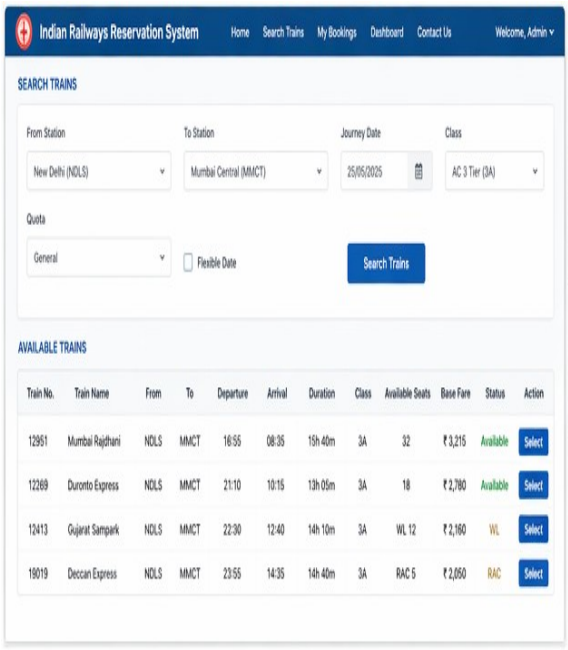
📄 Deliverables

1. PrimeFaces XHTML pages
2. ManagedBeans (well-structured)
3. REST API integration code
4. Dashboard implementation
5. Migration documentation (RichFaces → PrimeFaces)
6. Performance optimization notes
7. Screenshots of all modules

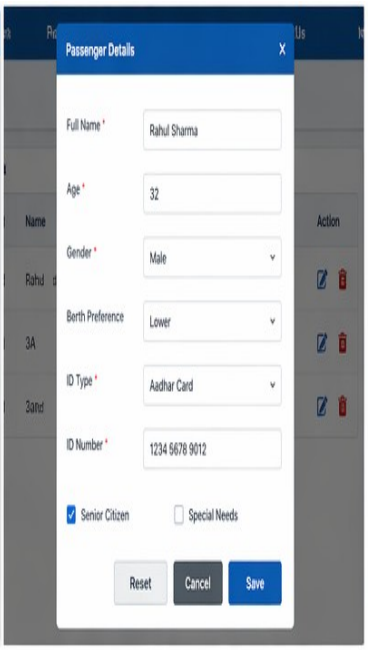
🔥 Final Insight

This single project forces you to demonstrate:

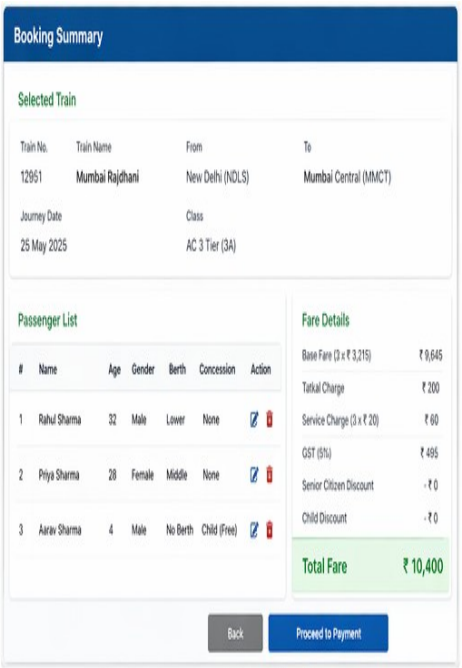
- Core PrimeFaces components
- AJAX lifecycle + partial updates
- REST API integration
- Advanced UI (charts, wizard, upload)
- Validation & UX
- Migration strategy
- Performance tuning



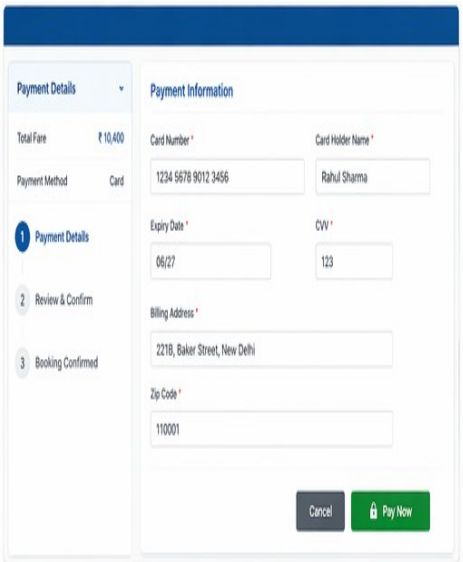
1. Train Search & Availability



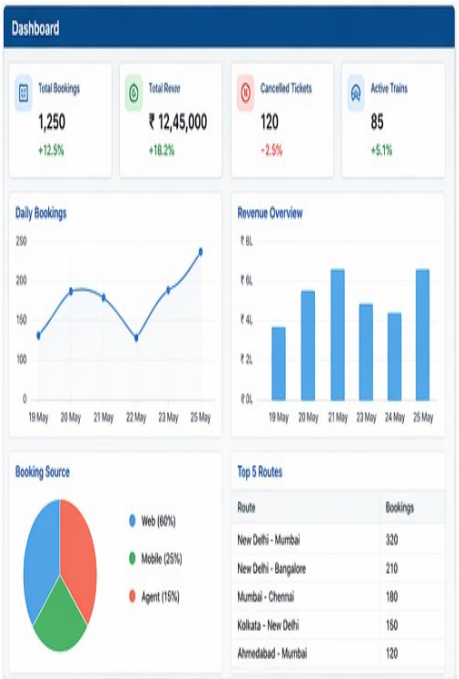
2. Passenger Management (Dialog)



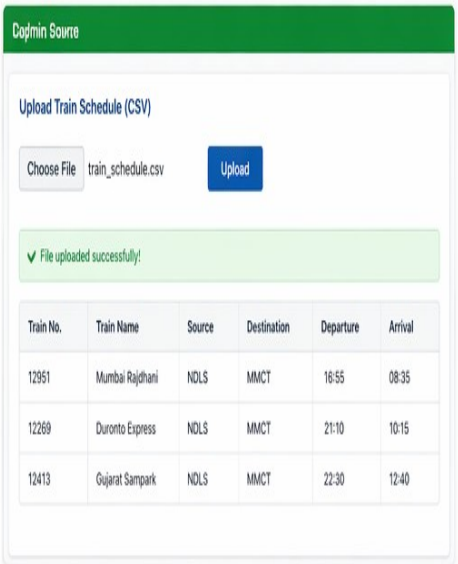
3. Booking Summary & Fare Calculation



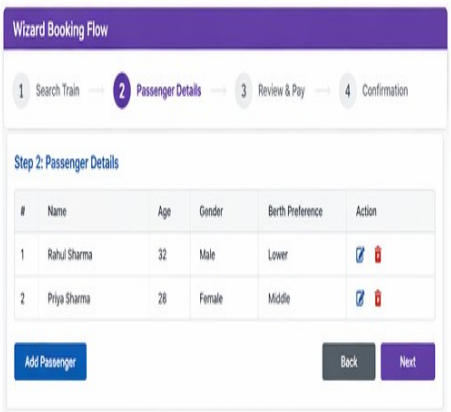
4. Payment Module



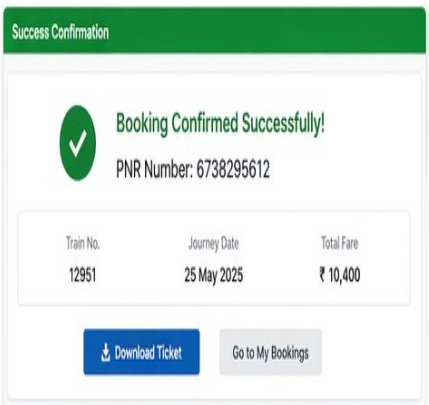
5. Admin Dashboard with Charts



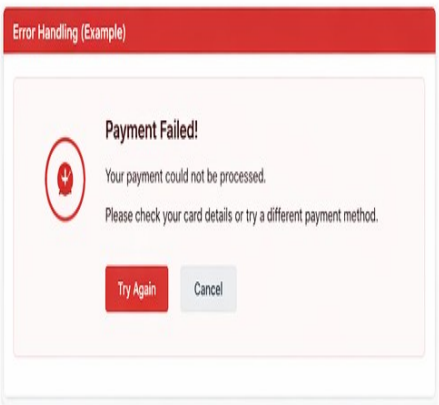
6. File Upload (Train Schedule)



7. Wizard Booking Flow



8. Booking Confirmation



9. Error Handling & User Feedback